Pontificia Universidad JAVERIANA Colombia

# LLMOps y despliegue

Semana 12 · El asistente de ACME como API
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

**Sergio Alberto Mora Pardo - H1 2027**
GitHub · LinkedIn · Repositorio del curso

# Tópicos Avanzados en Analítica II

Estructura del curso

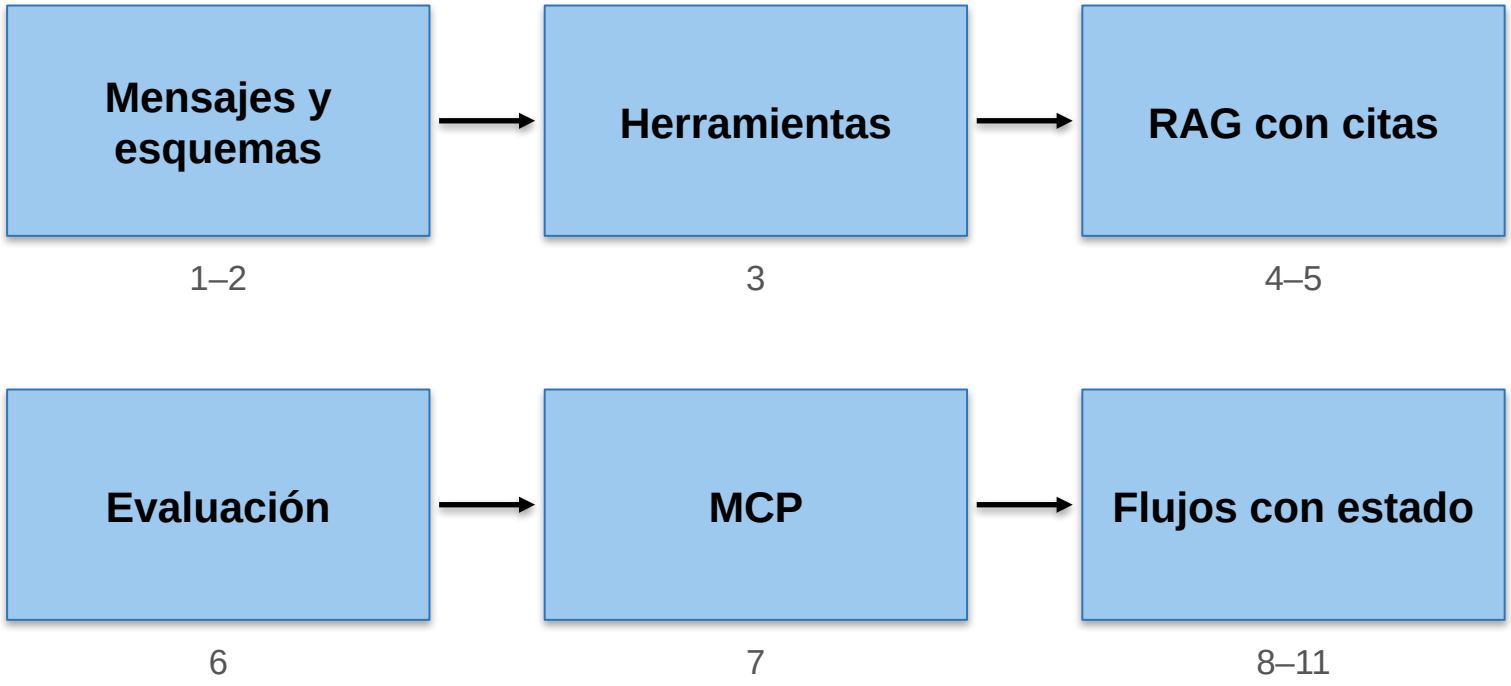
- Fundamentos con LangChain — Semanas 1–3 ✓
- RAG y evaluación — Semanas 4–6 ✓
- Model Context Protocol — Semana 7 ✓
- LangGraph y patrones agénticos — Semanas 8–11 ✓
- **LLMOps y despliegue** — Semana 12

## Semana 12 · LLMOps y despliegue

- Del notebook a un servicio
- Un agente que junta las piezas del curso
- FastAPI, `thread_id` y PostgreSQL
- Streaming con SSE
- LangGraph Studio y CI
- Operar: costos, trazas, evaluación y seguridad

# Lo que construimos en el curso

Doce semanas, un mismo asistente




Esta semana lo empaquetamos como un servicio que otros pueden usar.

# Del notebook a un servicio

El problema de la semana

**Qué cambia**

1. Muchas personas al tiempo, cada una con su conversación.
2. La memoria debe sobrevivir a un reinicio del servidor.
3. La respuesta debe empezar a mostrarse antes de terminar.
4. Hay que saber cuánto cuesta, qué falló y si la calidad se mantiene.

Pontificia Universidad JAVERIANA Colombia

# El agente de ACME

src/agents/acme.py

# Un agente con las piezas del curso

src/agents/acme.py

## Dos herramientas

`search_policies`: RAG con citas (semana 4).
`check_credit`: la cartera del servidor MCP (semana 7).

```
def make_graph(*, checkpointer=None):
    return create_agent(
        build_model(),  # claude-haiku-5-5
        tools=[search_policies, check_credit],
        system_prompt=SYSTEM_PROMPT,
        checkpointer=checkpointer,
    )

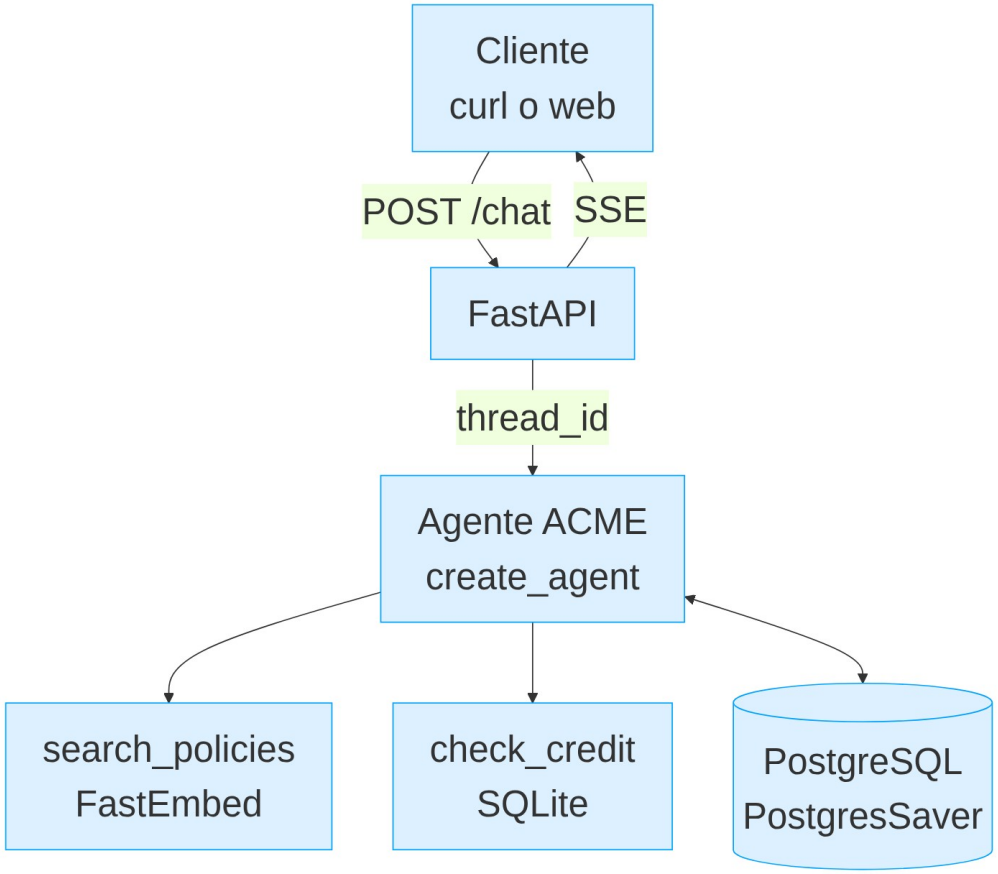
agent = make_graph()  # el grafo "acme" de langgraph.json
```

# La arquitectura

API local con memoria persistente

## Un hilo por conversación

El `chat_id` de la URL es el `thread_id` del checkpointer.



# Los endpoints

src/api/main.py

| Endpoint | Qué hace |
|---|---|
| `GET /health` | Comprueba que el servicio responde |
| `POST /chat/{chat_id}` | Responde y devuelve el historial guardado |
| `POST /chat/{chat_id}/stream` | Envía la respuesta por SSE mientras se genera |
| Mismo `chat_id` | Continúa la conversación, incluso tras reiniciar la API |

# Probar la API

README · Local support chat API

## Dos turnos, un hilo

El segundo turno entiende «lo» porque el hilo está en PostgreSQL.

```
docker compose up -d postgres
uv run --no-sync uvicorn api.main:app --port 8000

curl http://127.0.0.1:8000/chat/demo-thread \
  -H 'Content-Type: application/json' \
  -d '{"message":"Soy el cliente C-3307, ¿cuánto tengo vencido?"}'

curl -N http://127.0.0.1:8000/chat/demo-thread/stream \
  -H 'Content-Type: application/json' \
  -d '{"message":"¿Y todavía puedo reestructurarlo?"}'
```

# Streaming con SSE

src/api/main.py

**Mostrar antes**

1. `stream_mode="messages"` entrega los fragmentos del modelo mientras se generan.
2. Se filtran los del nodo `model`: la salida de las herramientas no llega al cliente.
3. Cada evento `data:` lleva un JSON con el campo `content`.
4. El primer texto llega antes; el tiempo total es el mismo.

Pontificia Universidad JAVERIANA Colombia

# Operar

LLMOps

# Qué vigilar en producción

LLMOps

| Dimensión | Cómo se mide en el curso |
|---|---|
| Costo | Tokens de entrada y salida por llamada; US$ por corrida |
| Calidad | Experimentos de LangSmith con dataset, código y juez (semana 6) |
| Trazas | LangSmith con `tracing_context` y un presupuesto de trazas |
| Regresiones | CI con pruebas sin modelo y arranque de `langgraph dev` |
| Seguridad | Herramientas de solo lectura, aprobación humana, claves fuera del código |

# Lo que esta demo no hace

Antes de exponerla

**Límites conocidos**

1. No tiene autenticación ni control de dueño por conversación.
2. No serializa turnos simultáneos sobre el mismo `chat_id`.
3. Las credenciales de PostgreSQL son de demostración y el servicio escucha en loopback.
4. Cada llamada usa un modelo de pago: medir antes de abrirla a más usuarios.

Pontificia Universidad JAVERIANA Colombia

# Laboratorio

API local y LangGraph Studio

# ¿Qué haremos?

Laboratorio

**Nuestro recorrido**

1. Levantar PostgreSQL con Docker Compose y la API con uvicorn.
2. Conversar con dos `chat_id` distintos y reiniciar la API entre turnos.
3. Abrir el mismo agente en LangGraph Studio (`acme`).
4. Inspeccionar los checkpoints guardados con `docs/sql/checkpoints.sql`.

# Las ideas que nos llevamos del curso

Cierre

**Criterios de construcción**

1. El modelo es un componente: la aplicación prepara el contexto y comprueba la salida.
2. Las respuestas se sustentan en herramientas y documentos citados.
3. Elegir el patrón más simple; los controles van en el código.
4. Medir calidad, latencia y costo antes de decidir.

# Para seguir leyendo

Lecturas de la semana

| Fuente | Qué revisar |
| --- | --- |
| Learning LangChain, cap. 9 y 10 | Despliegue y pruebas |
| AI Engineering, cap. 10 | Arquitectura y feedback de usuarios |
| Documentación de LangGraph | Persistence, deployment y Studio |




# ¡Gracias!

Todo el material del curso, los notebooks y las presentaciones están en el repositorio.

**Sergio Alberto Mora Pardo - H1 2027**

GitHub · LinkedIn · Repositorio del curso